

FOUNDATIONS OF DEEP LEARNING · 2025/2026

Audio Keyword Recognition Using Deep Learning

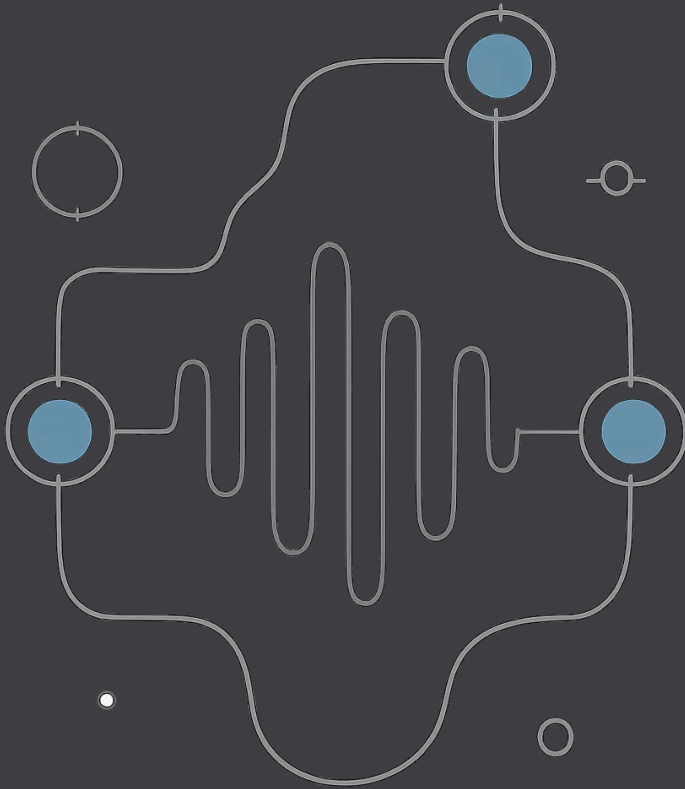
Group Members:

Abdul Sami — 939542 a.sami7@campus.unimib.it

Samuele — 949803 · s.urosevic@campus.unimib.it

Thilina Lakshan — 946895 t.millaniyage@campus.unimib.it

Tools: Python · TensorFlow · Keras · Librosa · Scikit-learn



PROBLEM STATEMENT

Bridging Raw Audio and Machine Understanding

Speech is the most natural human interface, yet machines cannot directly interpret raw audio waveforms. This project addresses the core challenge of mapping complex time-domain signals into meaningful categorical labels.

Convert

Transform spoken audio into machine-learnable representations

Classify

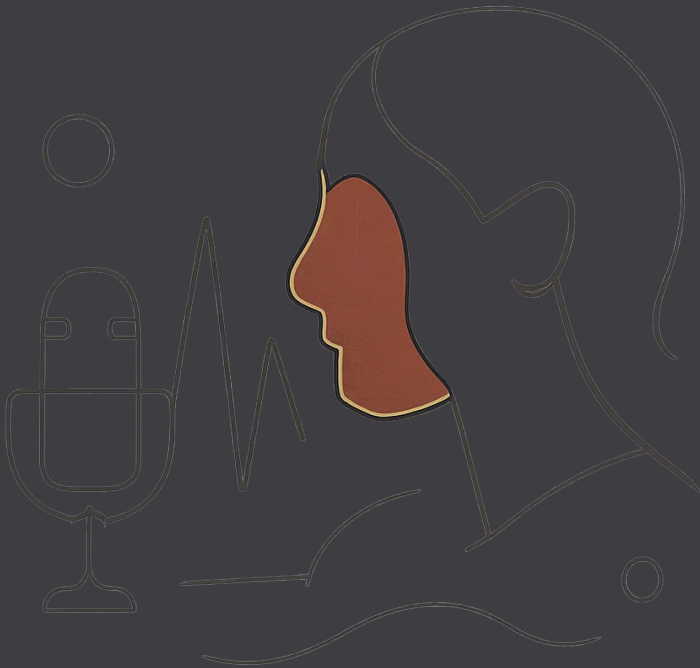
Identify short one-second speech commands accurately

Recognize

10 command words + silence + unknown class

Optimize

Build a robust, well-evaluated classifier



Google Speech Commands v0.01

Google Speech Commands v0.01 is a widely used benchmark for keyword spotting research, designed to help models learn how to recognize short spoken commands from real-world audio. It contains about 65,000 one-second recordings from thousands of speakers, making it a strong testbed for robust speech understanding.

Dataset Characteristics

- **~65,000** one-second audio recordings from thousands of speakers
- Benchmark dataset for keyword spotting and speech command recognition
- **10 command words:** yes, no, up, down, left, right, on, off, stop, go
- Two additional classes: **Silence** (background noise) and **Unknown** (non-target words)

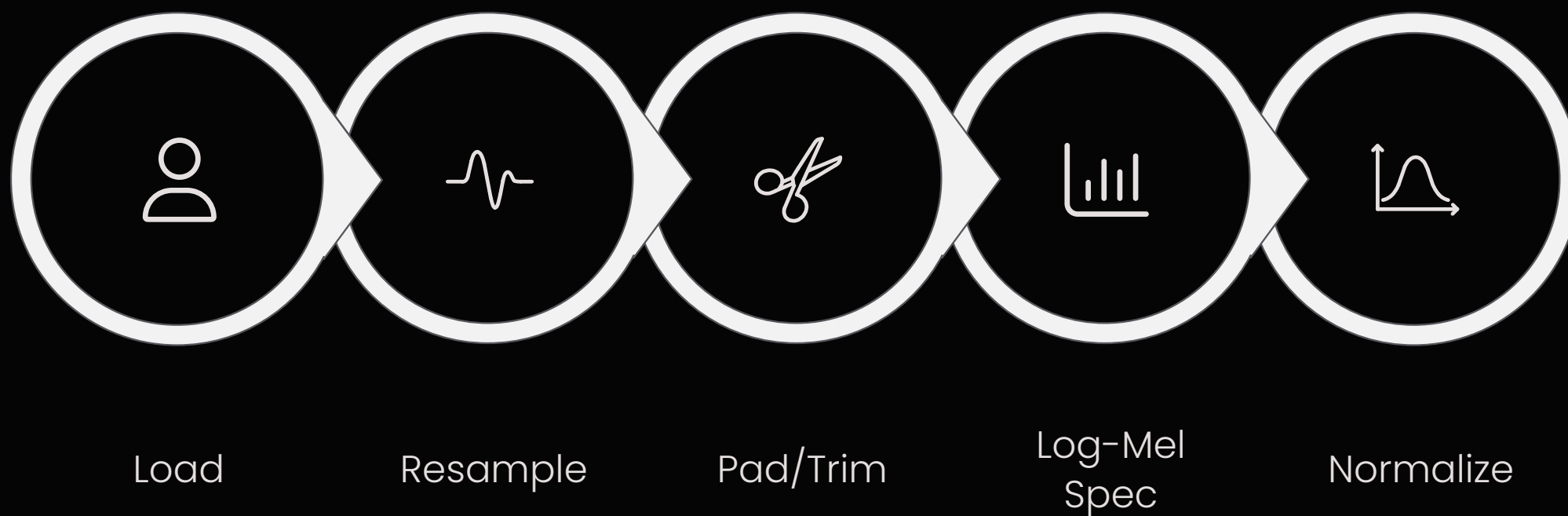
Split Breakdown

- **Total after preprocessing:** 64,727 samples
- **Training:** 45,334 samples
- **Validation:** 9,683 samples
- **Test:** 9,710 samples

A stratified split was used to preserve class distributions across the training, validation, and test sets. This helps ensure fair evaluation and supports reliable model training.

Audio Processing & Feature Extraction

Raw audio is a 1D time-series signal unsuitable for direct CNN input. A structured transformation pipeline converts it into a 2D time-frequency representation.

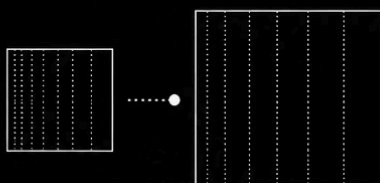


To ensure reproducibility, fixed random seeds were used during dataset splitting and model training.

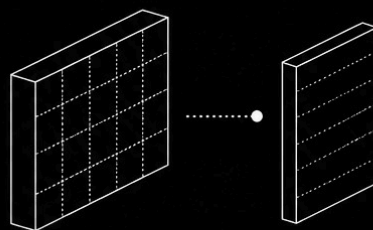
The final output is a normalized **64 × 32 × 1** matrix encoding the time-frequency energy distribution of the spoken command, ready for CNN ingestion.

Why Mel Spectrogram?

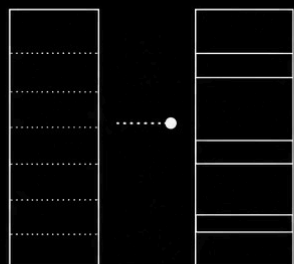
- 1 Mimics human auditory perception — non-linear frequency spacing matches ear sensitivity



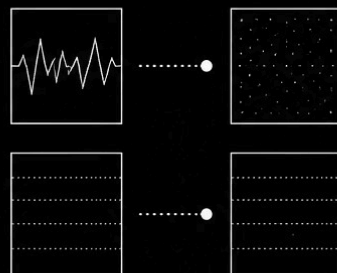
- 2 Compresses frequency information — reduces dimensionality while preserving speech content



- 3 Highlights speech-relevant bands — concentrates energy in formant regions



- 4 Reduces noise sensitivity — more robust than raw waveform



Key Insight

Human speech is far more distinguishable in the **frequency domain** than in the time domain. The Mel scale approximates how humans perceive pitch — linear below 1 kHz and logarithmic above — making it ideal for speech recognition tasks.

- ❏ The log (decibel) transformation further aligns the representation with human loudness perception.

Why CNN for Audio Classification?

CNN Advantages

- Spectrograms behave like **2D images** – CNNs excel at spatial pattern extraction
- Convolutional filters capture **local frequency patterns** corresponding to phonetic structures
- **Shared weights** reduce parameter complexity and improve generalization
- Hierarchical feature learning: edges → phonetic patterns → word-level features

CNN vs. RNN / LSTM

Faster Training

Parallelizable convolutions vs. sequential recurrence

Fixed-Length Inputs

Naturally suited to 1-second command clips

Stable Optimization

No vanishing gradient issues over time steps

End-to-End Project Pipeline

01

Load & Standardize Audio

Ingest 65,000 files, standardize to 1 second duration

02

Log-Mel Spectrogram Conversion

Transform waveforms using Librosa into time-frequency representation

03

Normalization

Scale spectrogram values to [0, 1]

04

Stratified Data Split

Divide into Train / Val / Test (70/15/15) using stratified sampling

05

CNN Training

Train custom CNN with Adam optimizer and sparse categorical cross-entropy loss

06

Training Callbacks

Apply EarlyStopping, ReduceLROnPlateau, and ModelCheckpoint

07

Model Evaluation

Assess accuracy, macro-F1, and confusion matrix on test set

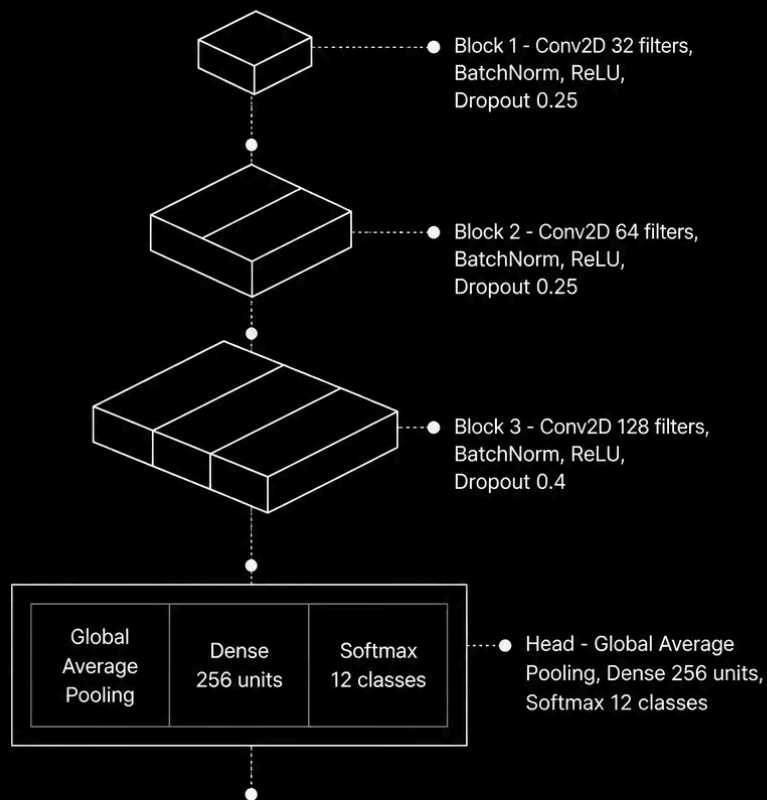
08

Transfer Learning Comparison

Benchmark against MobileNetV2 baseline

The pipeline standardizes audio to 1-second clips, converts them to Log-Mel spectrograms using Librosa, applies stratified sampling, trains a custom CNN with Adam optimization and training callbacks, and compares results against a MobileNetV2 transfer learning baseline.

CNN Model Architecture



Design Rationale

The architecture learns a **feature hierarchy** — from low-level edges to high-level phonetic representations. Three Conv2D layers (32 → 64 → 128 filters) progressively increase representational capacity.

- **Batch Normalization** stabilizes gradient flow and accelerates convergence
- **Dropout (0.25–0.4)** prevents overfitting on limited data
- **Global Average Pooling** reduces spatial dimensions without losing semantic information
- **Softmax (12 classes)** outputs normalized class probabilities
- **~129,000** parameters

Training Configuration

Hyperparameters

Optimizer

Adam · lr = 0.001

Loss Function

Sparse categorical
crossentropy

Batch Size

64 samples per batch

Epochs

15 training epochs

Callbacks

→ EarlyStopping

Halts training and restores the best weights when validation accuracy stops improving. Here it never triggered: validation accuracy was still rising at the final epoch (95.30%), so no plateau was reached and all 15 epochs ran — a sign the model had not yet saturated

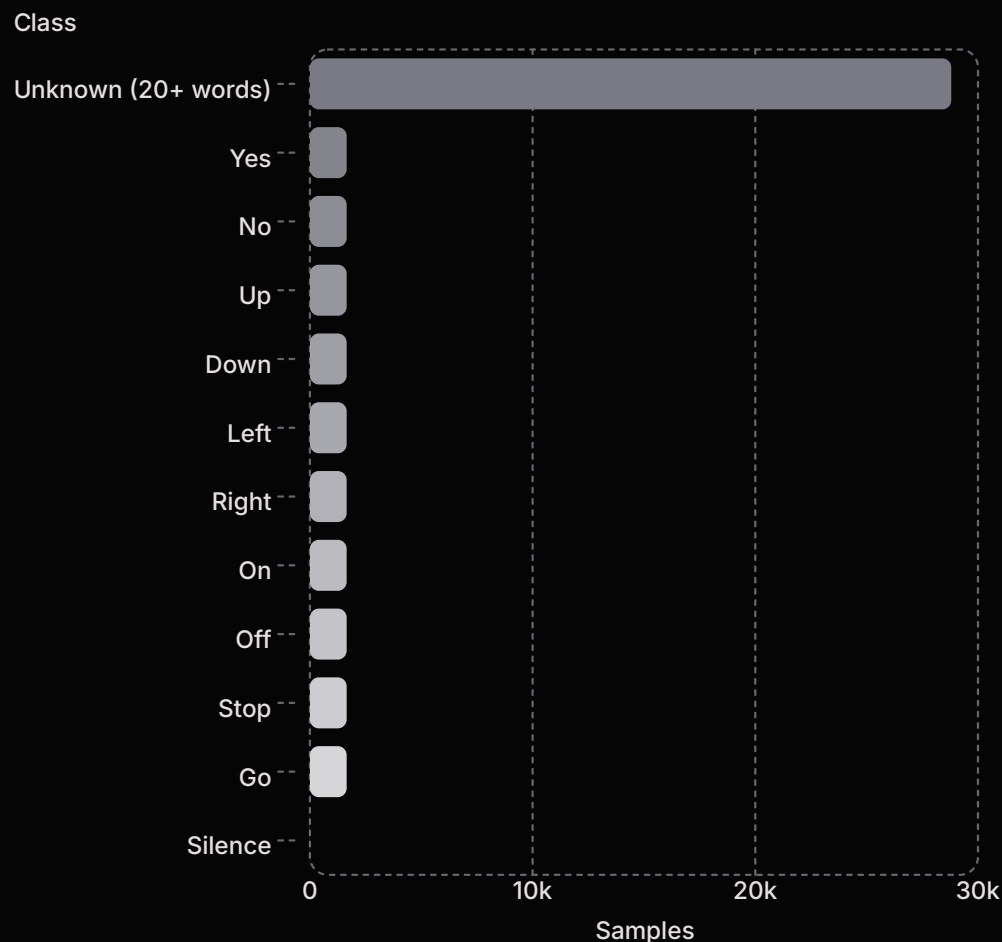
→ ReduceLROnPlateau

Lowers the learning rate when validation loss plateaus, allowing finer convergence in later epochs

→ ModelCheckpoint

Automatically saves the best model by validation accuracy — the weights restored at the end (epoch 15)

Data Distribution & Class Imbalance



Key Observations

- The 10 command classes are well balanced, with ~1,650–1,665 samples each
- Unknown dominates the training set: 28,744 samples (63.4%)** — roughly a 17:1 ratio against any single command class
- A trivial "always-unknown" classifier would already score ~63% accuracy, making raw accuracy a misleading metric
- Silence has only 4 training samples** — too few to learn from, and too few for the stratified split to allocate meaningfully to validation/test (just 1 silence sample lands in the test set)

Implications

- Strong imbalance requires evaluation **beyond accuracy**
- We adopt **Macro-F1**, which weights every class equally — command, unknown, and silence alike — exposing the failure on under-represented classes that accuracy would hide
- This directly explains the silence result ($F1 = 0.00$) and motivates the future-work fix of slicing background-noise recordings into 1-second windows

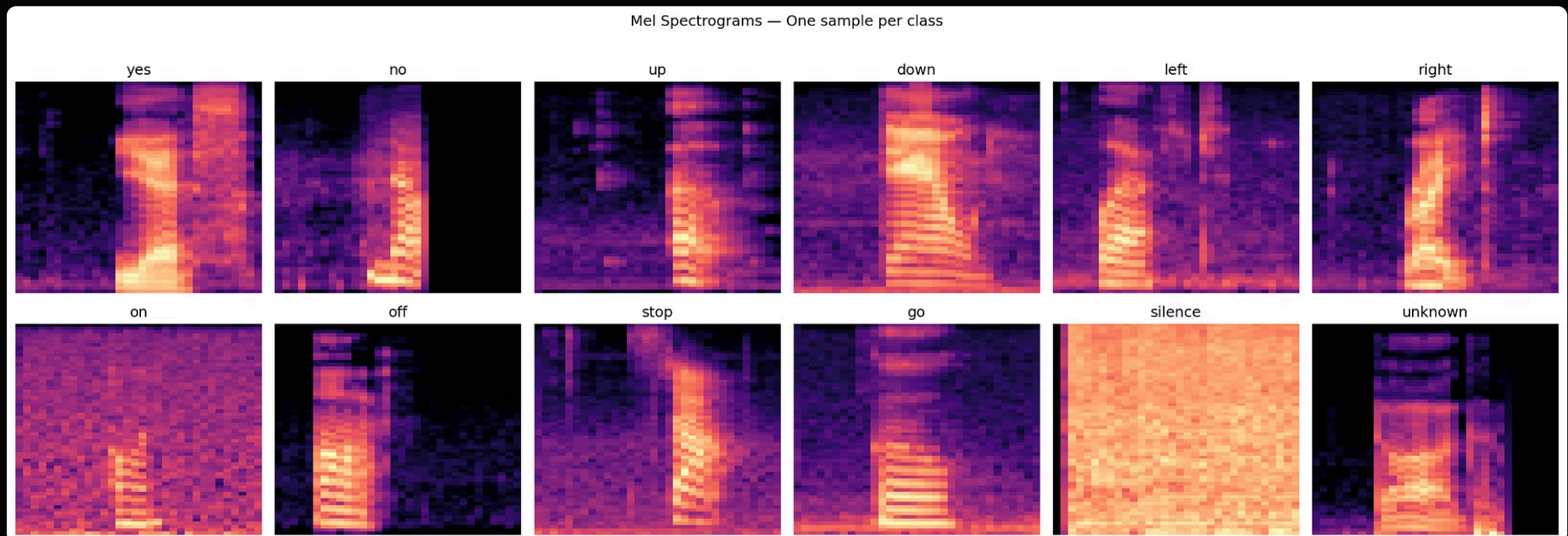
Spectrogram Visualization

Each class shows unique frequency-time signatures:

- “yes / go” → smooth vowel-dominant patterns
- “stop” → sharp high-energy burst (consonant cluster)
- “silence” → near-zero energy map
- “unknown” → high variability due to mixed words

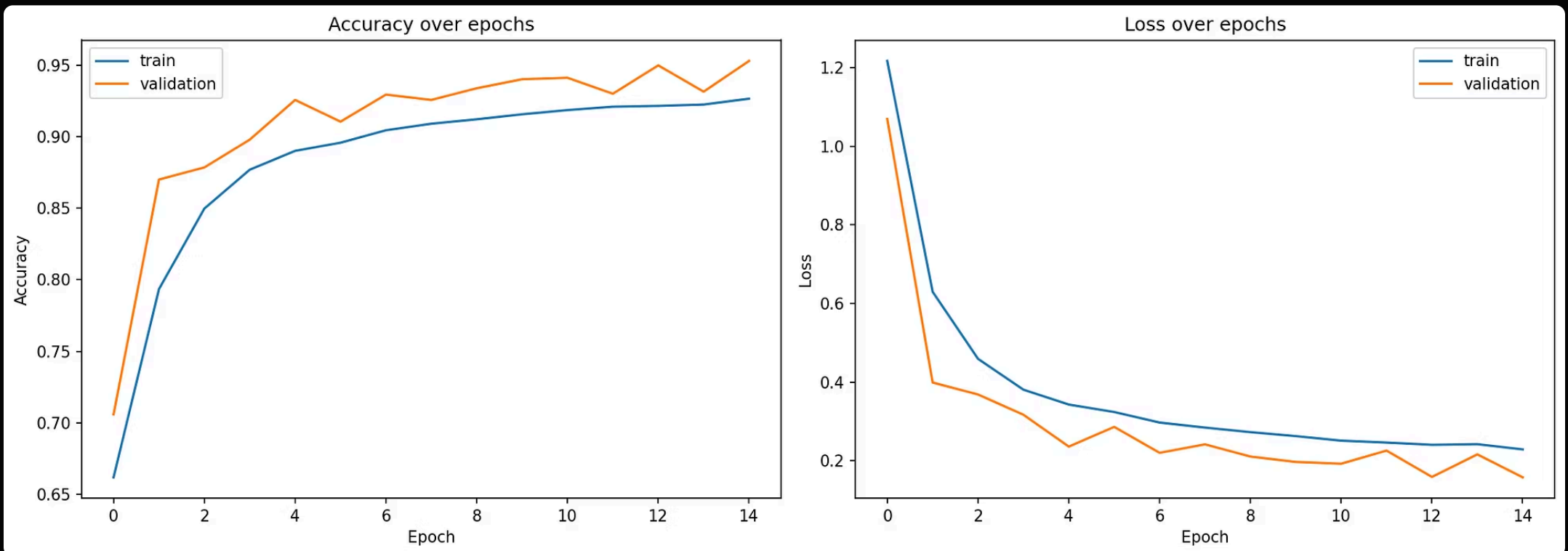
Key Insight:

This confirms that the command classes are visually separable. The unknown class aggregates many different words, so a single sample is not representative of its full variability



Training Curves Analysis

- Rapid convergence: Training accuracy 67% → 92.5%, validation accuracy reaching ~95% (peak 95.3% at the final epoch)
- Validation accuracy slightly higher than training (dropout active during training, disabled during validation) — indicates effective regularization
- Loss curves show consistent downward trend: Training loss ~1.22 → ~0.23, validation loss ~1.07 → ~0.16
- Temporary validation loss spike around epoch 5-6 is normal; validation accuracy remained high and recovered
- Small gap between training and validation metrics indicates good generalization with no significant overfitting
- Validation accuracy peaked at the final epoch (95.30%) and was still rising — early stopping never halted training, so more epochs would likely improve performance further



Confusion Matrix Analysis

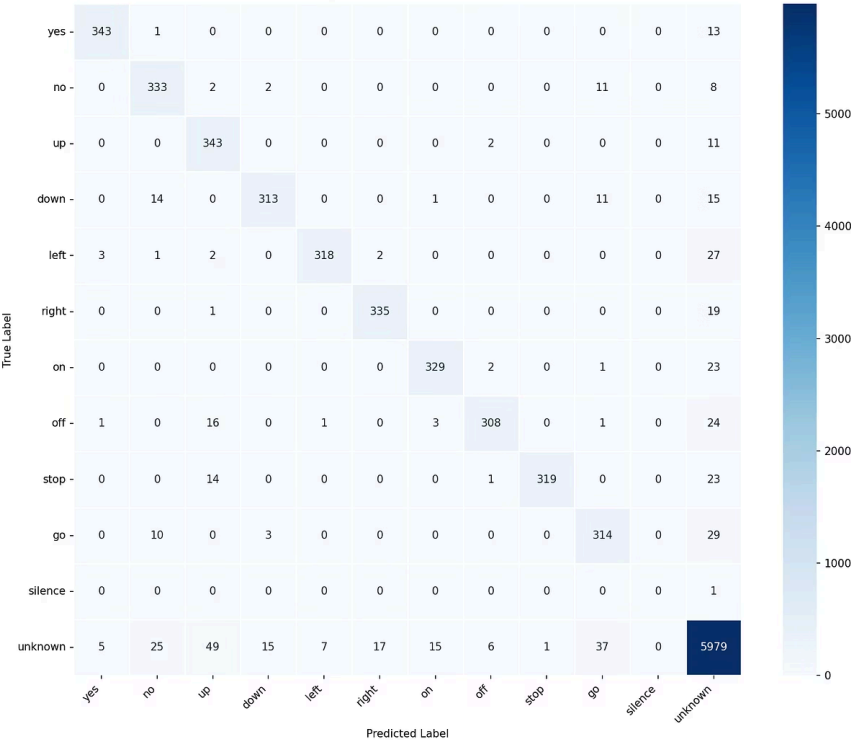
Key Observations:

- **Strong diagonal dominance** — every command class sits at 308–343 correct predictions out of ~355, so the model is right the vast majority of the time
- **Unknown is the error sink** — almost all command errors leak into the *unknown* column (go: 29, on: 23, stop: 23, off: 24, left: 27), not into other commands. The model rarely confuses one command for another; when unsure, it falls back to "unknown"
- **Command-to-command confusion is rare and phonetically motivated** — the few cases that exist are between acoustically similar words (e.g. off→up, down→no), consistent with shared phonetic structure rather than model failure
- **Silence is effectively untested** — only 1 test sample exists, and it lands in unknown; this reflects the 4-sample training limitation, not a classification weakness

Conclusion:

- The model is **robust and accurate**: errors are concentrated, not scattered
- The dominant failure mode — leaking into *unknown* — is the **safest possible error** for a keyword-spotting system: a missed command is far less costly than a command misfired as a different command
- Remaining command-to-command errors are phonetically explainable, not signs of an unstable model

Confusion Matrix — Test Set



MobileNet Comparison & Conclusion

Comparison:

To test whether transfer learning could beat our from-scratch CNN, we benchmarked against a frozen MobileNetV2 backbone.

- Custom CNN: **95.10%** accuracy · **0.85** macro-F1
- MobileNetV2 (frozen): **63.40%** accuracy · **~0.09** macro-F1

The 63.40% is not "lower accuracy" — it is collapse onto the majority class. Across all 20 epochs, MobileNetV2's training and validation accuracy stayed flat at exactly 0.6340 — the proportion of the *unknown* class. The model never learned to separate commands; it simply predicted "unknown" every time. This is precisely why macro-F1 matters: accuracy hides the collapse (63%), while macro-F1 exposes it (~0.065 vs our 0.85).

Why the Performance Gap?

- MobileNetV2 was pretrained on **ImageNet** (natural photographs) — its low-level filters expect edges and textures, not time-frequency energy patterns
- Spectrograms are **structurally unlike images**: the axes mean frequency and time, not space
- The **backbone was frozen**, so these mismatched features could not adapt to the audio domain
- With no useful gradient signal, the model defaulted to the statistically safest guess — the majority class

Final Conclusion:

Metric	Custom CNN	MobileNetV2
CNN Test Accuracy	95.10%	63.40%
Macro-F1	0.85	~0.065
Test Loss	0.1607	-
Best Val Accuracy	95.30%	63.40% (flat)
Total Parameters	129,676	-

Limitations & Future Work

Limitations

- **Class imbalance** — already detailed in the data analysis; handled via macro-F1, but it remains the defining constraint of this dataset
- **Silence unusable** — only 4 training samples → F1 = 0.00; addressed in future work below
- **Limited vocabulary** — only 10 command words; a real keyword-spotting system needs a broader, extensible set
- **Short clips (1s)** — minimal temporal context, which caps how much a sequence model could add over a CNN
- **Clean recordings** — v0.01 is recorded in quiet conditions; performance on noisy, real-world audio is untested and likely lower
- **MobileNetV2 frozen** — the transfer-learning baseline was not fine-tuned, so the comparison reflects frozen ImageNet features, not transfer learning's full potential

Future Work

- **Fix silence** — slice the background-noise recordings into 1-second windows to build a real silence class with enough samples to learn from
- **SpecAugment** — time/frequency masking, an augmentation designed for spectrograms (unlike image-style flips/crops, which are meaningless on time-frequency axes)
- **Noise-robustness testing** — evaluate on additive real-world noise before any deployment claim
- **Fine-tune MobileNetV2** — or, better, start from an *audio*-pretrained backbone (e.g. trained on AudioSet) rather than ImageNet
- **Transformer / longer context** — architectures suited to longer-range temporal dependencies, paired with longer clips
- **Real-time on-device inference** — the ~130k-parameter CNN is already lightweight enough to make this realistic

Despite these limitations, the from-scratch CNN delivers strong, well-evaluated performance — **95.1% accuracy · 0.85 macro-F1** — and clearly outperforms frozen transfer learning on spectrograms.